

For example, note in the following EXPLAIN statement how the actual time is missing from the output:

```
forumdb=> EXPLAIN (ANALYZE on, TIMING off) SELECT * FROM categories;
               QUERY PLAN
-----
Seq Scan on categories (cost=0.00..1.05 rows=5 width=68) (actual rows=5
loops=1)
Planning Time: 0.113 ms
Execution Time: 0.047 ms
(3 rows)
```

The SUMMARY option reports the total time spent in planning for the execution and the time spent for the query execution so that you can get an idea of how much effort the planner has used to find out the best execution plan.

The BUFFERS option, which defaults to off, provides information about the data buffers the query used to complete. For example, note how there is buffer-related information on the **Execution** node in the following query:

```
forumdb=> EXPLAIN (ANALYZE, BUFFERS on) SELECT * FROM posts;
               QUERY PLAN
-----
Seq Scan on posts (cost=0.00..31.04 rows=1004 width=71) (actual
time=0.006..0.100 rows=1004 loops=1)
  Buffers: shared hit=21
Planning Time: 0.061 ms
Execution Time: 0.171 ms
(4 rows)
```

The buffer information is not trivial to analyze and can be split into two parts: a prefix and a suffix.

The prefix can be any of the following:

- shared, meaning a PostgreSQL shared buffer, which is the database in-memory cache
- temp, meaning temporary memory (used for sorting, hashing, and so on)
- local, meaning temporary database objects space (for instance, temporary tables)

The suffix can be any of the following:

- hit, providing the number of memory successes
- read, providing the number of buffers read from the storage (therefore, not in the cache)
- dirtied, the number of buffers modified by the query
- written, the number of buffers removed from the PostgreSQL cache and written to disk
- lossy, the number of buffers that PostgreSQL has checked in memory in a second pass

Combining the prefix and the suffix provides information on the buffers. For example, in the previous query, the buffer line contained `shared hit=21`, which reads as “21 buffers have been successfully found in the database cache, no more operations on buffers are required.”

The WAL option provides information about the WAL usage of a writing statement. As an example, consider the following query, which adds a bunch of fake usernames to the users table:

```
forumdb=> EXPLAIN (ANALYZE on, WAL on, FORMAT yaml)
INSERT INTO posts( title, content, author, created_on, category )
SELECT 'A random post title ' || v, md5( v::text ), v%2 + 1, current_date
- v, v%5 + 1
FROM generate_series( 1, 100000 ) v;
```

QUERY PLAN

```
-----
- Plan:                                     +
  Node Type: "ModifyTable"                 +
...
  Actual Loops: 1                          +
  WAL Records: 506908                      +
  WAL FPI: 651                            +
  WAL Bytes: 46291062                     +
...

```

As you can see, the output reports information about the number of WAL records that have been generated (506908), the number of WAL **Full Page Images (FPIs)** (651), and the number of bytes written into the WAL logs (46291062). Therefore, the above statement has generated around 46 MB of WAL traffic.



WAL sizes will always be greater than the actual table sizes. For instance, in the above example, the data produced in the table is around 13 MB, while the WAL size is much greater and around 46 MB. The reason for this difference in size is that PostgreSQL must ensure data is safe in the WALs and useful for crash recovery; therefore, the WALs do not store only the content of the data but also metadata about how to properly restore such content.

Examples of query tuning

In the previous section, you learned how EXPLAIN can show the plan PostgreSQL will use to access the underlying data; it is now time to use EXPLAIN to tune some slow queries and improve performance.

This section will show you some basic concepts of the day-to-day usage of EXPLAIN as a powerful tool to determine where and how to instrument PostgreSQL in accessing data faster. Of course, query tuning is a very complex subject and often requires repeated trial-based optimization, so the aim of this section is not to provide you with in-depth knowledge about query tuning but rather a basic understanding of how to improve your own database and queries.

Sometimes, tuning a query involves simply rewriting it in a way that is more comfortable for – or better, more comprehensible to – PostgreSQL, but most often, query tuning means using an appropriate index to speed up access to the underlying data.

One important thing to take into account when query tuning is the *cache effect*: when PostgreSQL accesses some data, it loads the data into memory in the shared buffers. This memory area works as a cache; therefore, if PostgreSQL requires the same data again, it will pull it out of memory instead of going to the disk storage. The implication of this is that a query could be slow at first, but if executed a second time and, more in general, over and over, the time it takes could be smaller than the initial one. Therefore, when inspecting a slow query, try to execute it more and more in order to see how the caching effect can affect the time taken.

Our database is supposed to contain a thousand authors, each one publishing five hundred posts, for a grand total of half a million posts:

```
forumdb=> SELECT reltuples, pg_size_pretty( pg_relation_size( oid ) ),
relname FROM pg_class
WHERE relname IN ( 'posts', 'users' ) AND relkind = 'r';
reltuples | pg_size_pretty | relname
```

```

-----+-----+-----
500000 | 59 MB          | posts
1000   | 88 kB          | users

```



The Docker image for this chapter is already populated with the above amount of data. Please be careful, since such data will require you to have around 60 MB of free disk space.

Let's start with a simple example; we want to extract all the posts ordered by creation day, so the query is as follows:

```
SELECT * FROM posts ORDER BY created_on;
```

We can pass it to EXPLAIN to get an idea about how PostgreSQL will execute it:

```

forumdb=> EXPLAIN SELECT * FROM posts ORDER BY created_on;
              QUERY PLAN
-----
Sort  (cost=83838.92..85088.92 rows=500000 width=81)
   Sort Key: created_on
   -> Seq Scan on posts  (cost=0.00..12584.00 rows=500000 width=81)

```

As you can see, the first node to be executed is the sequential scan (the initial cost is 0), which is going to produce 500,000 tuples as output. Why a sequential scan? First of all, there is no filtering clause – we want to retrieve all the data stored in the table – and second, there is no access method on the table (there are no indexes).

Since we asked to sort the output, the following node to execute is a **Sorting** node, which produces the very same number of tuples as a result.

How much time does it take to complete the preceding query? EXPLAIN ANALYZE can help us answer that question:

```

forumdb=> EXPLAIN ANALYZE SELECT * FROM posts ORDER BY created_on;
              -----
              -----
              -----
Sort  (cost=83838.92..85088.92 rows=500000 width=81) (actual
time=304.283..393.395 rows=500000 loops=1)
   Sort Key: created_on

```

```

Sort Method: external merge  Disk: 51192kB
-> Seq Scan on posts  (cost=0.00..12584.00 rows=500000 width=81)
(actual time=0.045..63.449 rows=500000 loops=1)
Planning Time: 0.059 ms
Execution Time: 429.838 ms

```

The pure execution time is near half of a second. Is it possible to reduce the total amount of time by building a specific index on the `created_on` field:

```

forumdb=> CREATE INDEX idx_posts_date ON posts( created_on );
CREATE INDEX
forumdb=> EXPLAIN ANALYZE SELECT * FROM posts ORDER BY created_on;

QUERY PLAN
-----
Index Scan using idx_posts_date on posts  (cost=0.42..16887.80 rows=500000
width=81) (actual time=0.079..133.305 rows=500000 loops=1)
Planning Time: 0.203 ms
Execution Time: 162.143 ms
(3 rows)

```

The query is now running at almost one-third of the time required without the index, and, in fact, the query plan has changed from a sequential scan to an index scan with the freshly created new index.

Of course, the newly created index has a penalty in terms of storage space; as you can imagine, the increase in speed comes with an extra space cost that can be checked as follows:

```

forumdb=> SELECT pg_size_pretty( pg_relation_size( 'posts' ) ) AS table_
size,
      pg_size_pretty( pg_relation_size( 'idx_posts_date' ) ) AS index_size;
table_size | index_size
-----+-----
59 MB      | 3600 kB
(1 row)

```

Now, whether this extra disk space is too much or not depends on your resources and your final aim; in the preceding case, assuming you are executing the query quite often, the increased speed is justified by the additional space.

Let's now concentrate on a more typical query: finding out all the posts of a specific user in a specific period of time. The resulting query will be something like the following one, assuming a 2-day period:

```
SELECT p.title, u.username
FROM posts p
JOIN users u ON u.pk = p.author
WHERE u.username = 'fluca1978'
AND daterange( CURRENT_DATE - 20, CURRENT_DATE ) @> p.created_on::date
```

How does PostgreSQL execute the preceding query? Again, EXPLAIN can help us understand what the database thinks the best query plan is:

```
forumdb=> EXPLAIN
          SELECT p.title, u.username
          FROM posts p
          JOIN users u ON u.pk = p.author
          WHERE u.username = 'fluca1978'
          AND daterange( CURRENT_DATE - 20, CURRENT_DATE ) @> p.created_
on::date;

                                QUERY PLAN

-----
Gather  (cost=1008.30..13803.58 rows=2 width=26)
  Workers Planned: 2
    -> Hash Join  (cost=8.30..12803.38 rows=1 width=26)
          Hash Cond: (p.author = u.pk)
            -> Parallel Seq Scan on posts p  (cost=0.00..12792.33 rows=1042
width=15)
                  Filter: (daterange((CURRENT_DATE - 2), CURRENT_DATE) @>
(created_on)::date)
              -> Hash  (cost=8.29..8.29 rows=1 width=19)
                    -> Index Scan using users_username_key on users u
(cost=0.28..8.29 rows=1 width=19)
                          Index Cond: (username = 'fluca1978'::text)
```

The planner has chosen to fire up the parallel query execution: the top node is a **Gather** node, and thus acts as a synchronization point for parallel workers. In particular, the planner has decided to use two parallel processes to complete the query. The rightmost node, and therefore the first one being executed, is an **Index Scan** node on the users table. Results are hashed and, in the meantime, a **Parallel Seq Scan** is fired to extract data from the posts table. The results are joined and the **Gather** collects the final result set. The total execution time of this query can be easily obtained with **EXPLAIN ANALYZE**, which shows us that the query takes around 140 milliseconds to complete:

```

forumdb=> EXPLAIN ANALYZE
  SELECT p.title, u.username
  FROM posts p
  JOIN users u ON u.pk = p.author
  WHERE u.username = 'fluca1978'
  AND   daterange( CURRENT_DATE - 20, CURRENT_DATE ) @> p.created_
on::date;

QUERY PLAN
-----
Gather  (cost=1008.30..13803.58 rows=2 width=26) (actual
time=0.856..141.434 rows=20 loops=1)
  Workers Planned: 2
  Workers Launched: 2
    -> Hash Join  (cost=8.30..12803.38 rows=1 width=26) (actual
time=0.968..135.209 rows=7 loops=3)
      Hash Cond: (p.author = u.pk)
        -> Parallel Seq Scan on posts p  (cost=0.00..12792.33 rows=1042
width=15) (actual time=0.076..133.975 rows=6667 loops=3)
          Filter: (daterange((CURRENT_DATE - 20), CURRENT_DATE) @>
(created_on)::date)
            Rows Removed by Filter: 160000
        -> Hash  (cost=8.29..8.29 rows=1 width=19) (actual
time=0.053..0.054 rows=1 loops=3)
          Buckets: 1024  Batches: 1  Memory Usage: 9kB

```

```

-> Index Scan using users_username_key on users u
(cost=0.28..8.29 rows=1 width=19) (actual time=0.047..0.048 rows=1
loops=3)
      Index Cond: (username = 'fluca1978'::text)
Planning Time: 0.188 ms
Execution Time: 141.471 ms

```

What happens if we add an index to the author column of the posts table?

```

forumdb=> CREATE INDEX idx_posts_author ON posts( author );
CREATE INDEX
forumdb=> EXPLAIN ANALYZE
  SELECT p.title, u.username
  FROM posts p
  JOIN users u ON u.pk = p.author
  WHERE u.username = 'fluca1978'
  AND daterange( CURRENT_DATE - 20, CURRENT_DATE ) @> p.created_
on::date;

QUERY PLAN
-----
Nested Loop  (cost=8.45..1602.29 rows=2 width=26) (actual
time=0.145..0.926 rows=20 loops=1)
  -> Index Scan using users_username_key on users u  (cost=0.28..8.29
rows=1 width=19) (actual time=0.011..0.012 rows=1 loops=1)
        Index Cond: (username = 'fluca1978'::text)
  -> Bitmap Heap Scan on posts p  (cost=8.17..1593.98 rows=2 width=15)
(actual time=0.129..0.904 rows=20 loops=1)
        Recheck Cond: (author = u.pk)
        Filter: (daterange((CURRENT_DATE - 20), CURRENT_DATE) @> (created_
on)::date)
        Rows Removed by Filter: 480
        Heap Blocks: exact=500
  -> Bitmap Index Scan on idx_posts_author  (cost=0.00..8.17
rows=500 width=0) (actual time=0.059..0.059 rows=500 loops=1)

```

```

Index Cond: (author = u.pk)
Planning Time: 0.401 ms
Execution Time: 0.954 ms
(12 rows)

```

First of all, the query now runs sequentially because there are no **Gather** nodes: this means that now PostgreSQL is able to reduce, in advance, the result set to inspect. Moreover, the execution time is now less than a millisecond.

As you can see, the first executed node is an **Index Scan** node on the `posts` table, which is now used first, unlike in the parallel previous plan, and the reduced result set is then joined by means of a **Nested Loop** with the `authors` table. The usage of a **Nested Loop** confirms that PostgreSQL has been able to reduce the result set obtaining in advance only the needed tuples from the `posts` table.

It is also interesting to note that there is no longer any reason to keep the index on the `created_on` column, since the preceding query plan is not using it anymore.

How much space is required for the indexes now? Again, it is quite simple to check:

```

forumdb=> SELECT pg_size_pretty( pg_relation_size( 'posts' ) ) AS table_
size,
           pg_size_pretty( pg_relation_size( 'idx_posts_date' ) ) AS idx_date_
size,
           pg_size_pretty( pg_relation_size( 'idx_posts_author' ) ) AS idx_
author_size;

 table_size | idx_date_size | idx_author_size
-----+-----+-----
59 MB      | 3600 kB       | 3600 kB

(1 row)

```

Both the indexes require the same space, but as we already said, we can drop the date-based index since it is no longer required. In fact, even with a specific date clause, the index is not used anymore:

```

forumdb=> EXPLAIN ANALYZE
SELECT p.title, u.username
FROM posts p
JOIN users u ON u.pk = p.author
WHERE u.username = 'fluca1978'

```

```

AND    p.created_on::date = CURRENT_DATE -2;

                                         QUERY PLAN
-----
Nested Loop  (cost=8.45..1599.79 rows=2 width=26) (actual
time=0.132..0.737 rows=1 loops=1)
  -> Index Scan using users_username_key on users u  (cost=0.28..8.29
rows=1 width=19) (actual time=0.009..0.010 rows=1 loops=1)
        Index Cond: (username = 'fluca1978'::text)
  -> Bitmap Heap Scan on posts p  (cost=8.17..1591.48 rows=2 width=15)
(actual time=0.119..0.723 rows=1 loops=1)
        Recheck Cond: (author = u.pk)
        Filter: ((created_on)::date = (CURRENT_DATE - 2))
        Rows Removed by Filter: 499
        Heap Blocks: exact=500
        -> Bitmap Index Scan on idx_posts_author  (cost=0.00..8.17
rows=500 width=0) (actual time=0.053..0.053 rows=500 loops=1)
                Index Cond: (author = u.pk)
Planning Time: 0.237 ms
Execution Time: 0.763 ms

```

Identifying unused indexes is important because it allows us to reclaim disk space and simplifies the management and data insertion: remember that every time the table changes, the index has to be updated, and this also requires extra resources, such as time and disk space.

Therefore, as you can see, it is really important to analyze the queries your applications execute the most and identify whether an index can help improve the execution speed, but also remember that an index has an extra cost in both space and maintenance, so don't abuse the use of indexes.

But how can you identify unused indexes without even knowing about the ongoing queries?

Luckily, PostgreSQL provides you with detailed information about the usage of every index: the special `pg_stat_user_indexes` view provides information about how many times an index has been used and how. For example, to get information about the indexes over the `posts` table, you can execute something such as the following:

```

forumbd=> SELECT indexrelname, idx_scan, idx_tup_read, idx_tup_fetch FROM
pg_stat_user_indexes WHERE relname = 'posts';

```